PHP y bases de datos

En las primeras versiones de PHP, la única manera de acceder a un SGBD era a través de una librería o biblioteca específica, que contenía las funciones necesarias. Con el paso del tiempo, este enfoque fue mostrando sus limitaciones. El principal inconveniente de este enfoque es que cada extensión está estrechamente vinculada a cada SGBD y si se quiere cambiar de SGBD es necesario reescribir completamente la aplicación (los nombres de cada función son diferentes, el orden de los argumentos distinto, las funcionalidades disponibles son diferentes, etc.). Además en algunos casos, las extensiones no están mantenidas adecuadamente (bugs de seguridad, etc.).

Para poder escribir programas independientes del SGBD elegido, es necesario utilizar una capa de abstracción que permita acceder de una forma común a cualquier SGBD. Esta necesidad no es exclusiva de PHP, sino que afecta a cualquier aplicación que necesita trabajar con un SGBD y no tiene solución completa pues las diferencias entre los SGBD no permiten olvidarse de cuál estamos utilizando realmente. Una de estas capas de abstracción es ODBC (Open DataBase Connectivity), una API independiente del sistema operativo, lenguaje de programación o SGBD utilizada que se empezó a desarrollar en 1992. El inconveniente de ODBC es que introduce un elemento más en la cadena, el controlador ODBC del sistema operativo, lo que puede ralentizar el rendimiento.

La capa de abstracción propia de PHP se llama PDO (Php Data Objects), librería orientada a objetos e incluye controladores para acceder a las bases de datos más populares. Utilizando PDO no podemos olvidarnos completamente del SGBD utilizado, pero la mayor parte del código es independiente del SGBD y sólo en algunas partes del programa (en la conexión con el SGBD o en la creación de tablas, por ejemplo) el código es específico del SGBD.

¿Qué es PDO?

PDO (**PHP Data Objects**) es una librería de PHP para conectarse a bases de datos de forma orientada a objetos. Características principales:

- Es **independiente del motor** (MySQL, PostgreSQL, SQLite, Oracle, etc.).
- Usa una **API unificada**: el mismo código sirve para distintos motores.
- Ofrece **seguridad** mediante consultas preparadas.
- Es más **flexible** y moderno que `mysqli`.

PDO separa el código de aplicación del motor de base de datos.

Conexión con la base de datos

Para trabajar con una base de datos en nuestro programa, simplemente nos conectaremos con la base de datos creando un objeto PDO. La información necesaria para la conexión con la base de datos específica se incluye como uno o varios argumentos:

```
// CONEXIÓN CON LA BASE DE DATOS

$pdo = new PDO("informacion-para-la-conexion");
```

La información para la conexión consiste en primer lugar en el DSN (Data Source Name), que indica como mínimo el nombre de la base de datos (sqlite, mysql, pgsql, etc.) y su ubicación. En su caso, el segundo y tercer argumento son el nombre y contraseña del usuario de la base de datos.

Si por algún motivo PDO no puede conectarse con la base de datos, se genera una excepción. PDO permite elegir al usuario entre tres formas distintas de gestionar los errores encontrados al realizar las consultas (normalmente por errores de sintaxis de SQL):

- PDO::ERRMODE_SILENT

  En este modo, los errores de PDO no interrumpen la ejecución del programa, simplemente se guardan y se pueden consultar mediante los métodos errorCode() y errorInfo(). Si utilizamos este modo, nuestro programa tiene que encargarse de comprobar tras cada consulta si ha habido un error y tenerlo en cuenta.

- PDO::ERRMODE_WARNING

  En este modo, PDO además de guardar los errores genera un aviso de tipo E_WARNING que, dependiendo de la configuración de error reporting, se mostrará o no en pantalla.

- PDO::ERRMODE_EXCEPTION

  En este modo, PDO genera una excepción de clase PDOException. Si utilizamos este modo, nuestro programa tiene que gestionar las excepciones, porque si no el programa se interrumpe con un error E_ERROR.

Hasta PHP 8.0, publicado en 2020, el modo predeterminado era ERRMODE_SILENT, pero desde esa versión es ERRMODE_EXCEPTION.

Para conectar con base de datos deberíamos implementar el siguiente código:

```php
<?php

    $host = 'pruebaphp_db_1';
    $dbname = 'toor'; //Nombre de la base de datos
    $user = 'inma'; //Nombre del usuario
    $pass = 'inma'; //Password del usuario
    $port = 3306;

    // Crear una instancia de PDO para la conexión con SSL habilitado
    $dsn = "mysql:host=$host;port=$port;dbname=$dbname;charset=utf8mb4";
    $options = [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, // Manejo de errores
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC, // Modo de recuperación
        PDO::ATTR_EMULATE_PREPARES => false, // Desactiva la emulación de sentencias
preparadas
    ];

    // Conectar a la base de datos
    $connection = new PDO($dsn, $user, $pass, $options);
    // echo "Conexión exitosa a la base de datos.";
```

A partir de aquí, cuando queramos acceder a la base de datos utilizaremos la variable `$pdo` . Se recomeinda crear un fichero de configuración para almacenar los datos de conexión.

```php
// Fichero config.php
<?php

return [
    'db' => [
        'host' => 'php-db-1', //Nombre del contenedor del gestor de BD
        'dbname' => 'ejemploPost', //Nombre de la base de datos
        'user' => 'root', // Usuario
        'pass' => 'root' // contraseña
    ]
];
```

Podemos comprobar si funciona o no nuestra conexión

```php
<?php

try {
    require_once __DIR__ . '/connection.php';
    echo 'Conexión realizada correctamente.';
} catch (PDOException $exception) {
    echo $exception->getMessage();

}
```

En una aplicación web PHP habitual se crea una conexión y se reutiliza durante la petición. Al terminar, PHP libera los recursos si no quedan referencias. No debemos guardar el objeto PDO en la sesión ni abrir una conexión nueva por cada fila.

Consultas

Para realizar una consulta seguiremos los siguientes pasos:

```php
$statement = $connection->query(
    'SELECT id, titulo, cuerpo FROM posts ORDER BY id'
);
$posts = $statement->fetchAll(PDO::FETCH_ASSOC);

foreach ($posts as $post) {
    echo htmlspecialchars($post['titulo'], ENT_QUOTES, 'UTF-8');
    echo '<br>';
}
```

- Podemos **usar `query` cuando no hay parámetros externos**. Nos devuelve un objeto PDOStatement
- **`$statement`**: Es el objeto que representa la consulta ejecutada.
- **`fetchAll(PDO::FETCH_ASSOC)`**: Recupera los resultados de la consulta y los convierte en un array asociativo, con claves que corresponden a los nombres de las columnas.

Con fetchAll() recuperamos todos los datos de la consulta de una única vez, y con fetch() vamos a poder recuperar uno por uno los registros. ¿Cuál es el más recomendado?

La elección entre usar `fetchAll()` para obtener todos los datos de una consulta de una sola vez, o utilizar `fetch()` para procesar los registros uno por uno depende del **contexto** y de lo que estemos buscando en términos de **eficiencia** y **uso de memoria**.

Opción 1: Usar `fetchAll(PDO::FETCH_ASSOC)`

- **Ventajas**:
  - **Fácil de usar**: Obtenemos todos los datos de una sola vez en un array, lo que te permite procesar o manipular los datos en bloque.
  - **Ideal para pequeños conjuntos de datos**: Si sabemos que la cantidad de datos devueltos es pequeña o manejable, `fetchAll()` es una opción sencilla y directa.
  - **Más limpio para recorrer**: Al tener todos los datos en un array, podemos utilizar estructuras como `foreach` fácilmente y aplicar transformaciones o cálculos sin preocuparnos por la iteración manual.
- **Desventajas**:
  - **Consumo de memoria**: Si la consulta devuelve una gran cantidad de registros, `fetchAll()` carga todos los resultados en la memoria de una sola vez. Esto puede resultar ineficiente y, en casos extremos, provocar que la aplicación consuma mucha RAM o incluso cause un error de memoria.
- **Cuándo usarlo**:
  - Cuando el número de registros devueltos es pequeño o moderado y no afecta demasiado el consumo de memoria.
  - Cuando necesitamos trabajar con todos los datos a la vez, por ejemplo, para crear un archivo CSV, generar un informe completo, o trabajar con todos los resultados en un mismo paso.

Opción 2: Usar `fetch(PDO::FETCH_ASSOC)` en un bucle

- **Ventajas**:
  - **Mejor uso de memoria**: Con `fetch()`, solo un registro se carga en la memoria a la vez. Esto es más eficiente cuando trabajamos con grandes volúmenes de datos, ya que solo se procesa un registro por iteración.
  - **Procesamiento en tiempo real**: Puedes procesar los registros uno por uno mientras se están obteniendo. Esto es útil si necesitamos aplicar lógica de negocio o enviar los datos a otro sistema en cada iteración.
- **Desventajas**:
  - **Menos cómodo**: Al procesar los datos uno por uno, puede ser un poco más tedioso si necesitamos manipular todos los resultados de una vez.
  - **Menos eficiente en casos de pequeños conjuntos de datos**: Si tenemos pocos registros, procesarlos uno por uno puede no tener ventajas significativas y solo añadir

código adicional.

- **Cuándo usarlo**:
  - Cuando esperamos que la consulta devuelva un **gran número de registros**, como en informes con miles o millones de filas.

Ejemplo con `fetchAll()` (CUIDADO CON SU USO!):

```
$statement = $connection->query("SELECT * FROM clientes");
$result = $statement->fetchAll(PDO::FETCH_ASSOC);

foreach ($result as $row) {
    echo "ID: " . $row['id'] . "<br>";
    echo "Nombre: " . $row['nombre'] . "<br>";
    echo "Apellidos: " . $row['apellidos'] . "<br><br>";
}
```

Ejemplo con `fetch()` en un bucle:

```
$statement = $connection->query("SELECT * FROM clientes");

while ($row = $statement->fetch(PDO::FETCH_ASSOC)) {
    echo "ID: " . $row['id'] . "<br>";
    echo "Nombre: " . $row['nombre'] . "<br>";
    echo "Apellidos: " . $row['apellidos'] . "<br><br>";
}
```

Recomendación:

1. **Pocos datos (pequeñas tablas, consultas específicas)**: Usa `fetchAll()`.
2. **Muchos datos (consultas que devuelven miles o millones de filas)**: Usa `fetch()`. Si no sabes cuántos registros devolverá tu consulta en un futuro y no los necesitas todos a la vez, usa `fetch()`.

Consultas preparadas

Las consultas preparadas son más seguras y evitan **inyección SQL**.

```php
$query = "SELECT * FROM clientes WHERE id = :id";
$statement = $db->prepare($query);
$statement->bindParam(':id', $clienteId, PDO::PARAM_INT); //el parámetro con el tipo no es obligatorio
$statement->execute();
$client = $statement->fetch(PDO::FETCH_ASSOC);
```

- `:id` es un marcador de posición en la consulta.
- `bindParam()` asocia el valor de una variable PHP (`$clienteId`) con el marcador de la consulta.
- `execute()` ejecuta la consulta con el valor proporcionado.
- Luego podremos recorrerrlo con fetch o con fetchAll.

Siempre usar consultas preparadas cuando haya datos introducidos por el usuario.

Otra forma de proporcionar los datos

```php
$statement = $connection->prepare(
    'SELECT id, titulo FROM posts WHERE titulo LIKE :search'
);
$statement->execute(['search' => '%' . $search . '%']);
$posts = $statement->fetchAll();
```

En este fragmento, $search es un texto previamente obtenido y validado. execute([...]) es una forma cómoda de pasar valores. Si necesitas indicar expresamente un tipo, como un entero para LIMIT, utiliza bindValue().

Manejo de errores:

Para manejar los errores deberemos usar try catch

```php
try {
    $stmt->execute();
} catch (PDOException $e) {
    echo "Error en la consulta: " . $e->getMessage();
}
```

Insertar modificar y eliminar

INSERT y el identificador generado

```php
$title = 'Nueva publicación';
$content = 'Contenido de la publicación.';
$statement = $connection->prepare(
    'INSERT INTO posts (titulo, cuerpo) VALUES (:title, :content)'
);
$statement->execute(['title' => $title, 'content' => $content]);
$postId = (int) $connection->lastInsertId();
```

MySQL genera el id porque es AUTO_INCREMENT. No enviamos un id con valor 0 ni calculamos el siguiente con MAX(id) + 1. Recuperamos el identificador desde la misma conexión, justo después de insertar.

UPDATE con una condición

```php
$statement = $connection->prepare(
    'UPDATE posts SET titulo = :title, cuerpo = :content WHERE id = :id'
);
$statement->bindValue(':title', $title);
$statement->bindValue(':content', $content);
$statement->bindValue(':id', $postId, PDO::PARAM_INT);
$statement->execute();
$affectedRows = $statement->rowCount();
```

MySQL devuelve normalmente el número de filas realmente modificadas: 0 puede significar que el id no existe o que los valores ya eran los mismos. No equivale necesariamente a un error.

DELETE con una condición

```php
$statement = $connection->prepare('DELETE FROM posts WHERE id = :id');
$statement->bindValue(':id', $postId, PDO::PARAM_INT);
$statement->execute();
$affectedRows = $statement->rowCount();
```

En esta eliminación por clave primaria, 1 indica una fila eliminada y 0 que no había una fila con ese id.

Qué indica execute

Con ERRMODE_EXCEPTION, un fallo de SQL o una restricción incumplida genera una PDOException. Ejecutar correctamente la sentencia no garantiza que haya modificado una fila:

debemos interpretar el resultado según la operación.